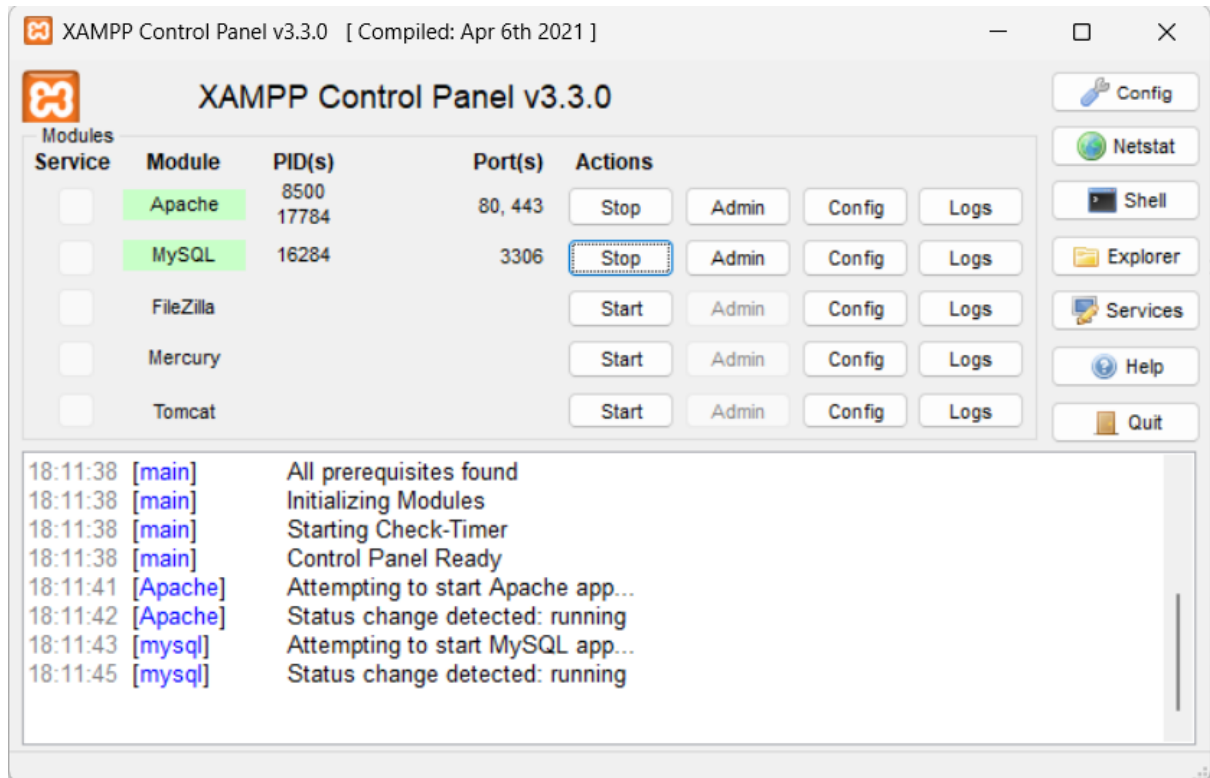


Session 1:-

Q1.Install MySQL Workbench on your computer, connect to the local MySQL server, and take a screenshot of the default databases shown in the Schemas panel.



Q2.Create a new database called InstaClone in MySQL Workbench, then create a table named Users with columns: user_id (INT, primary key), username (VARCHAR), email (VARCHAR), and followers_count (INT).

```
1 CREATE TABLE users(  
2     user_id int PRIMARY key AUTO_INCREMENT,  
3     username varchar(30),  
4     email varchar(30),  
5     followers_count int  
6 );
```

Q3. Insert 3 sample users into the Users table you created, using realistic Instagram-style usernames and follower counts.

				user_id	username	email	followers_count
☐				1	vedant	NULL	1500
☐				2	vijay	NULL	1860
☐				3	mahesh	NULL	1600
	☐	Check all	With selected:				

Q4. Create another table in the InstaClone database called Posts with columns: post_id (INT, primary key), user_id (INT), caption (VARCHAR), and post_date (DATE). Add a foreign key from Posts.user_id to Users.user_id to establish a relationship.

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0354 seconds.)

CREATE TABLE posts( post_id int PRIMARY key AUTO_INCREMENT, user_id int , caption varchar(30), post_date date, FOREIGN
key(user_id) REFERENCES users(user_id) );

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

```
1 CREATE TABLE posts(  
2     post_id int PRIMARY key AUTO_INCREMENT,  
3     user_id int ,  
4     caption varchar(30),  
5     post_date date,  
6     FOREIGN key(user_id) REFERENCES users(user_id)  
7 );
```

Q5. Write a short comparison (3-4 lines each) of MySQL, PostgreSQL, Oracle, and SQLite, focusing on where each one is commonly used (for example: web apps, mobile apps, enterprise systems, etc.).

Sessions 2:-

Q1. Open MySQL Workbench or CLI and write a CREATE TABLE statement to create a table called 'restaurants' with columns: id (INT, primary key, auto-increment), name (VARCHAR(100)), location (VARCHAR(100)), and rating (DECIMAL(2,1)).

```
1 CREATE table restaurants(  
2     r_id int PRIMARY key AUTO_INCREMENT,  
3     r_name varchar(30),  
4     r_location varchar (50),  
5     r_rating decimal (2,1)  
6 );|
```

Q2. Use the ALTER TABLE command in MySQL to add a new column 'average_cost' (INT) to your 'restaurants' table.

Hint: Remember to specify the data type when adding a new column.

```
1 ALTER TABLE restaurants ADD average_cost int ;
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 r_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 r_name	varchar(30)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3 r_location	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 r_rating	decimal(2,1)			Yes	NULL			Change Drop More
☐	5 average_cost	int(11)			Yes	NULL			Change Drop More

☐ Check all With selected: Browse Change Drop Primary Unique Index Spatial
 Fulltext

Q3. Write a SQL statement to DROP the 'restaurants' table you created earlier. Double-check your command before running it to avoid accidental data loss.

```
1 DROP TABLE restaurants;
```

Q4. Create a new table called 'playlists' for a music app like Spotify, with columns: playlist_id (INT, primary key, auto-increment), title (VARCHAR(50)), created_by (VARCHAR(50)), and created_at (DATE).

```

1 CREATE TABLE playlist(
2     playlist_id int PRIMARY KEY AUTO_INCREMENT,
3     title varchar(30),
4     created_by varchar(50),
5     created_at date
6 );

```

Sessions 3:-

Q1. Create a SQL table called Restaurants with columns: id (INT, auto-increment), name (VARCHAR), cuisine (VARCHAR), rating (DECIMAL), and city (VARCHAR). Insert 5 sample restaurants into the table, each with a different cuisine and rating.

				r_id	r_name	r_cuisine	r_rating
☐				1	spice villa	Indian	4.60
☐				2	pizza house	Itaillan	4.90
☐				3	Burger point	fast food	1.80
☐				4	Ocen grill	west indian	4.50
☐				5	Blue laggon	mexican	2.50


☐ Check all
 With selected:
  Edit
  Copy
  Delete
  Export

Q2. Write a SQL UPDATE statement to change the rating of the restaurant named 'Spice Villa' to 4.7 in the Restaurants table.

```
1 UPDATE restaurant
2 set r_rating = 4.7
3 WHERE r_name ='spice villa';
```

<

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Q3. Delete all restaurants from the Restaurants table where the rating is less than 3.5 using a SQL DELETE statement.

```
1 DELETE FROM restaurant
2 WHERE r_rating <3.5;
```

				r_id	r_name	r_cuisine	r_rating
☐				1	spice villa	Indian	4.70
☐				2	pizza house	Itaillan	4.90
☐				4	Ocen grill	west indian	4.50
	☐ Check all	With selected:					
☐ Show all	Number of rows: 25	Filter rows: Search this table	Sort by key: None				

Q4. Use a SELECT query with WHERE and LIMIT to fetch the top 2 restaurants in the city 'Ahmedabad' with a rating above 4.0 from the Restaurants table.
Hint: Combine WHERE for city and rating, and LIMIT to restrict the number of results.

Sessions :-4

Q1. Create a table called Playlist with columns: id, song_name, artist, genre, and play_count. Insert at least 5 sample records representing your favorite songs from Spotify.

```

1 CREATE TABLE Playlist(
2     id int PRIMARY key AUTO_INCREMENT,
3     song_name varchar(30),
4     artist varchar(20),
5     genre varchar(10),
6     play_count int
7 );

```

					id	song_name	artist	genre	play_count
☐	Edit	Copy	Delete		1	Shape of You	Ed Sheeran	Pop	150
☐	Edit	Copy	Delete		2	Believer	Imagine Dragons	Rock	220
☐	Edit	Copy	Delete		3	Kesariya	Arijit Singh	Romance	180
☐	Edit	Copy	Delete		4	Blinding Lights	The Weeknd	Pop	300
☐	Edit	Copy	Delete		5	Levitating	Dua Lipa	Pop	250

☐ Check all With selected: Edit Copy Delete Export

Q2. Write a SELECT query to display only the song_name and artist from the Playlist table, but show the artist column as 'Singer' using an alias.

✓ Showing rows 0 - 4 (5 total, Query took 0.0005 seconds.)

```
SELECT song_name, artist AS Singer FROM Playlist;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

					song_name	Singer
☐	Edit	Copy	Delete		Shape of You	Ed Sheeran
☐	Edit	Copy	Delete		Believer	Imagine Dragons
☐	Edit	Copy	Delete		Kesariya	Arijit Singh
☐	Edit	Copy	Delete		Blinding Lights	The Weeknd
☐	Edit	Copy	Delete		Levitating	Dua Lipa

☐ Check all With selected: Edit Copy Delete Export

Q3. Write a SELECT query to find all songs in the Playlist table where the genre is 'Pop' and play_count is greater than 100. Sort the results by play_count in descending order.

```
1 SELECT *
2 FROM Playlist
3 WHERE genre = 'Pop'
4     AND play_count > 100
5 ORDER BY play_count DESC;
```

Extra options

				id	song_name	artist	genre	play_count	1		
☐		Edit		Copy		Delete	4	Blinding Lights	The Weeknd	Pop	300
☐		Edit		Copy		Delete	5	Levitating	Dua Lipa	Pop	250
☐		Edit		Copy		Delete	1	Shape of You	Ed Sheeran	Pop	150

</

Q4. Use the COUNT aggregate function to find out how many songs in your Playlist table belong to the genre 'Hip-Hop'.

```
1 SELECT COUNT(*) AS hip_hop_song_count
2 FROM Playlist
3 WHERE genre = 'Hip-Hop';
4
```

hip_hop_song_count
2

Q5. Write a SELECT query that shows the total number of plays (SUM of play_count) for each genre in your Playlist table, grouped by genre.
Hint: Use GROUP BY with the SUM function to aggregate play counts by genre.

```
1 SELECT genre, SUM(play_count) AS total_plays
2 FROM Playlist
3 GROUP BY genre;
```

Session 5:-

Q1.Create two tables in SQL: influencers (with influencer_id as PRIMARY KEY and name) and posts (with post_id as PRIMARY KEY, influencer_id as FOREIGN KEY, and caption). Insert at least 3 influencers and 2 posts for each influencer.

```
1 CREATE TABLE influencers (  
2     influencer_id INT PRIMARY KEY,  
3     name VARCHAR(50)  
4 );|
```

```
1 CREATE TABLE posts (  
2     post_id INT PRIMARY KEY,  
3     influencer_id INT,  
4     caption VARCHAR(255),  
5     FOREIGN KEY (influencer_id) REFERENCES influencers(influencer_id)  
6 );|
```

Extra options

				influencer_id	name
☐	 Edit	 Copy	 Delete	1	Aarav Sharma
☐	 Edit	 Copy	 Delete	2	Priya Patel
☐	 Edit	 Copy	 Delete	3	Rohan Mehta

 ☐ Check all With selected:  Edit  Copy  Delete  Export

Extra options

				post_id	influencer_id	caption
☐	Edit	Copy	Delete	101	1	Morning workout routine!
☐	Edit	Copy	Delete	102	1	Healthy breakfast ideas.
☐	Edit	Copy	Delete	103	2	Traveling to Goa!
☐	Edit	Copy	Delete	104	2	Sunset at the beach.
☐	Edit	Copy	Delete	105	3	Learning SQL is fun.
☐	Edit	Copy	Delete	106	3	New coding project completed.

☐ Check all
 With selected:
 Edit
 Copy
 Delete
 Export

Q2. Write an SQL query using INNER JOIN to display each post's caption along with the name of the influencer who posted it, based on the tables you created.

```

1 SELECT p.caption, i.name
2 FROM posts p
3 INNER JOIN influencers i
4 ON p.influencer_id = i.influencer_id;
5

```

caption	name
Morning workout routine!	Aarav Sharma
Healthy breakfast ideas.	Aarav Sharma
Traveling to Goa!	Priya Patel
Sunset at the beach.	Priya Patel
Learning SQL is fun.	Rohan Mehta
New coding project completed.	Rohan Mehta

Q3. Write an SQL query using LEFT JOIN to show all influencers and their posts, including influencers who haven't posted anything yet. If an influencer has no posts, display 'No Posts' in the caption column.
Hint: Use IFNULL or COALESCE to handle NULL values in the result.

```
1 SELECT
2     i.name,
3     COALESCE(p.caption, 'No Posts') AS caption
4 FROM influencers i
5 LEFT JOIN posts p
6 ON i.influencer_id = p.influencer_id;
7 |
```

name	caption
Aarav Sharma	Morning workout routine!
Aarav Sharma	Healthy breakfast ideas.
Priya Patel	Traveling to Goa!
Priya Patel	Sunset at the beach.
Rohan Mehta	Learning SQL is fun.
Rohan Mehta	New coding project completed.

Q4. Write an SQL query using RIGHT JOIN to list all posts and the corresponding influencer's name, ensuring that even posts without a matching influencer_id (if any) are shown.

```
1 SELECT
2     p.post_id,
3     p.caption,
4     IFNULL(i.name, 'Unknown Influencer') AS influencer_name
5 FROM influencers i
6 RIGHT JOIN posts p
7 ON i.influencer_id = p.influencer_id;
```

post_id	caption	influencer_name
101	Morning workout routine!	Aarav Sharma
102	Healthy breakfast ideas.	Aarav Sharma
103	Traveling to Goa!	Priya Patel
104	Sunset at the beach.	Priya Patel
105	Learning SQL is fun.	Rohan Mehta
106	New coding project completed.	Rohan Mehta

Sessions 6:-

Q1. reate a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

user_id	order_count
101	3
102	2
103	1

Q2. Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

```
1 SELECT r_id,  
2 AVG(rating) AS rt  
3 FROM restaurent  
4 GROUP BY r_id  
5 HAVING AVG (rating)>4.0;
```

←T→ ▼				r_id	rt
☐	 Edit	 Copy	 Delete	2	4.700000
☐	 Edit	 Copy	 Delete	6	4.500000
☐	 Edit	 Copy	 Delete	7	4.800000
☐	 Edit	 Copy	 Delete	9	5.000000

Q3. suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.

```
SELECT r_id FROM restaurent WHERE payment>2000;
```

←T→				r_id
☐	 Edit	 Copy	 Delete	2
☐	 Edit	 Copy	 Delete	6



☐ Check all

With selected:

 Edit

 Copy

 Delete

 Export

Q4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.
 Hint :< /strong> Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query .< /em>

```
SELECT movie FROM restaurant WHERE rating > ( SELECT AVG(rating) FROM restaurant );
```

←T→

movie

☐

 Edit

 Copy

 Delete

☐

 Edit

 Copy

 Delete

jungle

☐

 Edit

 Copy

 Delete

dhurandhar-2

☐

 Edit

 Copy

 Delete

rajan



☐ Check all

With selected:

 Edit

 Copy

 Delete

 Export

Sessions 7:-

Q1. Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.

```
1 CREATE TABLE Restaurant (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     location VARCHAR(100) NOT NULL,  
5     cuisine VARCHAR(50)  
6 );
```

Q2. Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).

```
1 CREATE TABLE FoodOrder (  
2     order_id INT PRIMARY KEY,  
3     restaurant_id INT,  
4     user_id INT,  
5     order_total DECIMAL(10,2),  
6     FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
7 );
```

Q3. Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.

Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

```

1 CREATE TABLE SpotifyUser (
2     user_id INT PRIMARY KEY,
3     username VARCHAR(50) UNIQUE,
4     email VARCHAR(100) NOT NULL,
5     subscription_type VARCHAR(20)
6 );

```

Q3. Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.
Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

```

1 INSERT INTO SpotifyUser (user_id, username, email, subscription_type)
2 VALUES
3 (1, 'music01', 'music01@gmail.com', 'Premium'),
4 (2, 'master', 'master45@gmail.com', 'Free'),
5 (3, 'rockfan', 'rockfan99@gmail.com', 'Premium'),
6 (4, 'star', 'star@gmail.com', 'Student'),
7 (5, 'djvibes', 'djvibes@gmail.com', 'Family'),
8 (6, 'popstar22', 'popstar22@gmail.com', 'Premium');

```

				user_id	username	email	subscription_type
☐		Edit		Copy		Delete	1 music01 music01@gmail.com Premium
☐		Edit		Copy		Delete	2 master master45@gmail.com Free
☐		Edit		Copy		Delete	3 rockfan rockfan99@gmail.com Premium
☐		Edit		Copy		Delete	4 star star@gmail.com Student
☐		Edit		Copy		Delete	5 djvibes djvibes@gmail.com Family
☐		Edit		Copy		Delete	6 popstar22 popstar22@gmail.com Premium

☐ Check all
 With selected:
 Edit
 Copy
 Delete
 Export

Q4. Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

```
1 CREATE VIEW TopSpendersView AS
2 SELECT
3     s.username,
4     f.order_total
5 FROM FoodOrder f
6 INNER JOIN SpotifyUser s
7 ON f.user_id = s.user_id
8 WHERE f.order_total > 1000;
```

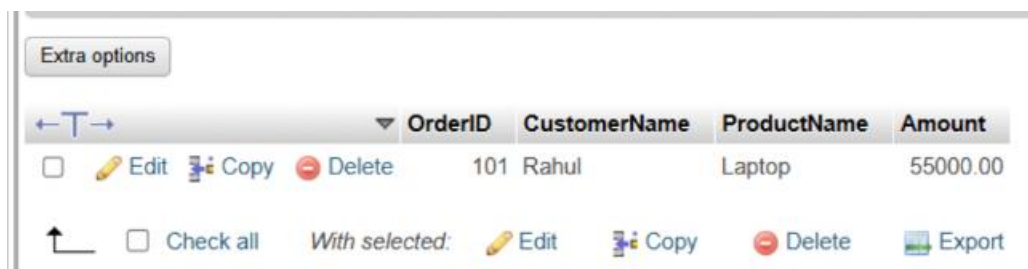
Q5. Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.

Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.

```
1 SELECT username, order_total
2 FROM TopSpendersView
3 WHERE order_total > 2000
4 ORDER BY order_total DESC;
```

Session 9:-

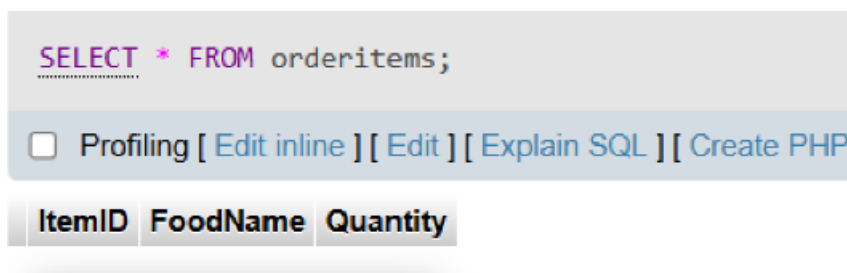
1. Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.



The screenshot shows a database management interface. At the top, there is a button labeled "Extra options". Below it, a table is displayed with the following columns: OrderID, CustomerName, ProductName, and Amount. The table contains one row with the values: 101, Rahul, Laptop, and 55000.00. Below the table, there are several icons and labels: a checkbox, "Edit", "Copy", "Delete", "Check all", "With selected:", "Edit", "Copy", "Delete", and "Export".

OrderID	CustomerName	ProductName	Amount
101	Rahul	Laptop	55000.00

2. Simulate a Zomato-style food order process: insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.

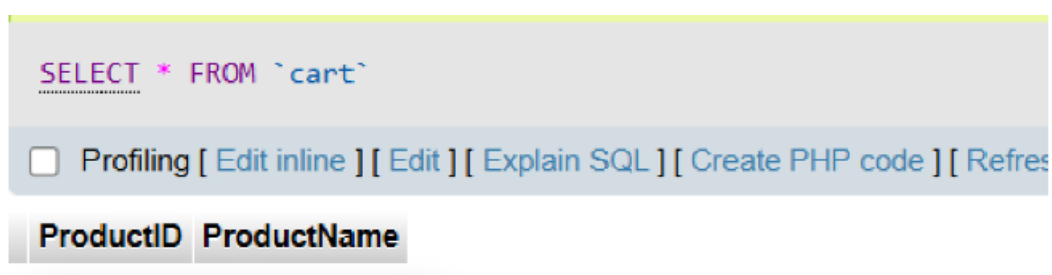


The screenshot shows a database management interface. At the top, there is a text area containing the SQL query: `SELECT * FROM orderitems;`. Below the text area, there are several buttons: "Profiling", "Edit inline", "Edit", "Explain SQL", and "Create PHP". Below the buttons, there is a table structure with the following columns: ItemID, FoodName, and Quantity.

```
SELECT * FROM orderitems;
```

ItemID	FoodName	Quantity
--------	----------	----------

3. In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.



The screenshot shows a database management interface. At the top, there is a text area containing the SQL query: `SELECT * FROM `cart``. Below the text area, there are several buttons: "Profiling", "Edit inline", "Edit", "Explain SQL", "Create PHP code", and "Refresh". Below the buttons, there is a table structure with the following columns: ProductID and ProductName.

```
SELECT * FROM `cart`
```

ProductID	ProductName
-----------	-------------

4. Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.



The screenshot shows a database management interface. At the top, there is a text area containing the SQL query: `SELECT * FROM `wallet``.

```
SELECT * FROM `wallet`
```

← T →		UserID		Balance	
☐	Edit	Copy	Delete	1	4300.00
	☐ Check all	With selected:		Edit	Copy
				Delete	Export

5. Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error.
 Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.

← T →		expense_id		user_id		amount	category	expense_date
☐	Edit	Copy	Delete	1	1	1000.00	Food	2026-07-05
	☐ Check all	With selected:		Edit	Copy	Delete	Export	

Mini project:-

Q1. Create a MySQL table called playlists to store Spotify-style music playlists, with columns: playlist_id (primary key), user_id (foreign key), name (varchar), and created_at (datetime). Write the CREATE TABLE statement.

```

1 CREATE TABLE playlists (
2     playlist_id INT AUTO_INCREMENT PRIMARY KEY,
3     user_id INT,
4     name VARCHAR(100) NOT NULL,
5     created_at DATETIME NOT NULL,
6     FOREIGN KEY (user_id) REFERENCES Users(user_id)
7 );

```

Q2. Insert three sample records into the playlists table: one for a user with a 'Workout Mix', one with a 'Chill Vibes', and one with a 'Top Hits' playlist. Use realistic data for each field.

```
1 INSERT INTO playlists (user_id, name, created_at)
2 VALUES
3 (1, 'Workout Mix', '2026-07-05 08:15:00'),
4 (2, 'Chill Vibes', '2026-07-05 14:30:00'),
5 (3, 'Top Hits', '2026-07-05 19:45:00');
```

Q3. Write an SQL UPDATE statement to rename the 'Chill Vibes' playlist to 'Evening Chill' for a specific user_id.

```
1 UPDATE playlists SET name="evening chill" WHERE user_id=1;
```

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		playlist_id	user_id	name	created_at
☐	Edit Copy Delete	5	1	evening chill	2026-07-01 13:31:30
☐	Edit Copy Delete	6	2	chill vibes	2026-07-01 13:31:30
☐	Edit Copy Delete	7	3	top hill	2026-07-01 13:31:30

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Q4. Write a DELETE statement to remove a playlist named 'Workout Mix' for a given user_id from the playlists table.
Hint: Make sure your WHERE clause targets only the intended row.

```
1 DELETE FROM playlists WHERE name="workout mix" and user_id=3;
```

Q5. Create a stored procedure named GetMonthlyPlaylistCount that takes a user_id and a month (as an integer) as input, and returns the total number of playlists the user created in that month.
Hint: Use the MONTH() function in your WHERE clause.

```
2  
3 CREATE PROCEDURE GetMonthlyPlaylistCount(  
4     IN p_user_id INT,  
5     IN p_month INT,  
6     OUT p_total_playlists INT  
7 )  
8 BEGIN  
9     SELECT COUNT(*) INTO p_total_playlists  
10    FROM playlists  
11    WHERE user_id = p_user_id  
12          AND MONTH(created_at) = p_month;  
13 END $$  
14  
15 DELIMITER ;
```